实验 2（新版 IC 开发实验）进展汇报

2026 年 10 月 21 日

今天做了什么

原计划今天只是跑测试。但开跑以后发现两个问题：一是第一版 IC 的检查环节基本没起作用；二是拿最强的模型做比较，几乎拉不开差距，比不出 IC 的价值。所以今天其实做了四件事：

1. **替换实验模型：**IC 组和裸跑模型统一为 GPT-5.6-luna-medium-fast。
2. **优化 IC 泛用性：**按测试中暴露的问题调整了 IC 的设计，其中一部分已经实现并验收，开始生效。 弱模型的"偷懒"由程序把关。 检查任务做成填表的形式，填得是否完整、是否真的测到了，由程序判定，不靠模型自觉。
3. **新增完整的轮内验收能力：**这是今天最主要的提高准确性的成果，相较于上版只看 SDD 回传的 PRD-check 流程，现在通过把四种能力分开并实现：发现（知道哪里不对）、定位（知道是哪个对象、哪句原文）、修复（改对）、不弱化已有能力。**上一版中存在的“系统和模型都遗漏的开发项、模型对题目逻辑理解出错但程序通过，始终无人发现”的错误现象在新版中显著降低。**
   现版本在待审产物生成后，子模型对照需求逐条检查，把问题写成纠错单交回去修。从之前的失分产物中生成离线测试，观察到有缺陷的程序修一次就能接近满分，例如 191 修到 218（满分 219），每次只需要几分钟，token 消耗低。

文中几个说法：

说法	含义
强模型	和“SDD 单独执行”用的同一个模型、同一推理档（GPT6-luna-high-fast）
降一档模型	更便宜的上一代模型，中等推理档；明天正式对比用它(GPT5.6-luna-medium-fast)
直接写	不加任何管理，把每轮需求原文直接交给模型去写
官方测试	评测方的测试，只用来打分，从不反馈给系统防作弊

一、IC 的正向成果

1. IC 内部验收+回修把有缺陷的产物修到接近满分。从第 1 轮的产出里挑了 4 份有缺陷的程序，由 IC 独立找出问题、写成纠错单，交给降一档模型修一次：

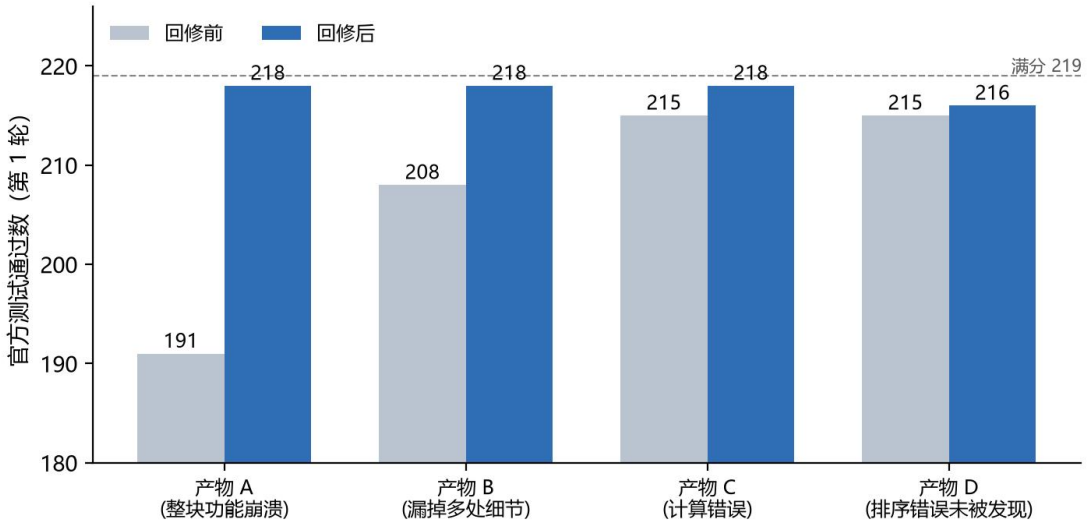


图 1　一次回修前后的官方测试通过数

产物	修前 → 修后	修好	改坏	回修耗时
A 整块功能崩溃	191 → 218	28	1	3.6 分钟
B 漏掉多处细节	208 → 218	11	1	2.3 分钟
C 计算错误	215 → 218	4	1	1.3 分钟
D 排序错误未被发现	215 → 216	1	0	4 分钟

每次回修使用干净上下文，约 0.3–1M token，大部分是缓存复用。

- 纠错单写到的真实缺陷全部修对了。
- A、B、C 各改坏 1 个，都来自同一条原文本身有歧义的要求。因此定下规则：判断为“原文有歧义”的条目不进纠错单，单独记录。
- D 没修好，是因为检查用的测试数据没能暴露这个错误（只用了一位数，按字符排序和按数值排序结果一样）。这是后续要补的短板。

2. 独立检查能发现执行模型自己发现不了的错误。之前版本观察到执行模型的自测只覆盖它想到的东西。目前初版实现是，我们让另一个模型专门针对原文的每一条要求写测试输入（不写期望答案），实际运行后，再由第三个模型对照原文判断对错。用 11 份已知官方结果的程序（2 份满分，9 份有缺陷）测：

写测试的方式	19 处已知缺陷命中	满分程序上误报	写测试的 token

IC 选定：较小的独立模型，填表 + 代码核对	13	0	约 9 万
降一档模型，填表 + 代码核对	11	2	约 23 万
降一档模型，不加约束自由写	只写了 20 个粗测试	—	—
参考上限：更强模型自由写	17	3	—

另做了对照：旧做法“预先算好期望值再比对”，22 处缺陷只发现 10 处，换成两份程序互相比对也一样，上限受测试输入覆盖限制。

- 关键做法：模型只负责理解，完整度由代码量：每个测试必须真的用到它声称要测的值，一个测试最多测 4 条要求，不合格就退回重写。弱模型从 20 个粗测试（偷懒）变成 56 个细测试（精细），便宜的模型也能写出有效检查。

3. IC 会在交付前，先在和评分机器一样的环境里试跑一遍，避免"写对了却跑不起来"。

（1）不试跑会怎样：从第 5 轮起整轮得 0 分。
不加管理、直接让强模型写时，前 4 轮几乎都是满分。到了第 5 轮，模型在代码里用了一个需要联网下载的外部组件。评分机器不能联网，程序装不起来，所以从第 5 轮起每轮官方得分都是 0。模型自己发现不了这个问题，因为它写代码的环境能联网。降一档的模型也在第 5 轮出了同样的问题。

（2）加上试跑以后：一次就修好了。
我们只加了一个最简单的步骤：每轮交付前，在不联网的环境里编译一次，跑一遍模型自己写的测试，把报错交回给模型。模型看到报错后，一次就改好了。第 5 轮的得分从 0 变成 641（满分 680）。IC 本身就带有这一步。

轮次	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**
测试总数	219	481	535	639	680	716
裸模型直接写	219	479	533	637	**0**	**0**
加上试跑	191	447	501	605	**641**	**677**

（3）"加上试跑"那一行前 4 轮为什么反而低了约 30 分？与试跑无关。

- 这两行是同一个模型的两次独立运行。第 1 轮试跑没有发现任何问题，没给模型发过一条反馈，所以这一轮的做法和"直接写"完全一样。
- 差的 28 个测试，**是模型那一次碰巧把一整块功能写崩了（随机影响）**。同样的事在"直接写"的重复运行里也会发生，大约每 4–5 次一次，见图 3 里的 191 和 203。
- **这个错误后面几轮一直没人修，所以每轮都带着这约 30 分的差距。**

这说明试跑只能发现"跑不起来"，发现不了"跑得起来但算错了"。后一类问题要靠第 1 点的"独立检查 + 回修"：同一份写崩的程序，回修一次就从 191 修到了 218。

4. IC 交给执行方的任务内容本身不拖后腿，成本问题出在执行方式（早上发现，已优化待完整测试）。

第 1 轮，强模型	各次得分	模型调用次数	总 token / 次
直接写（需求原文，一次做完）	219、218、218、218	21–31	约 0.5–0.8M
IC 任务单，一次做完	218、217、203、215	20–47	约 0.4–1.4M
IC 第一版，分 8 步执行	215	81	约 3.6M

任务单本身和原文得分相当；分步执行的调用多约 2.6 倍、token 多约 4.5 倍，却没有带来更高分。因此改为“一次做完，事后检查并回修”。

二、多轮测试的观察

观察 1：长任务里，新出现的错误一直没人修。下图是每一轮结束时累计没通过的官方测试数（不计联网问题，只看程序逻辑）。

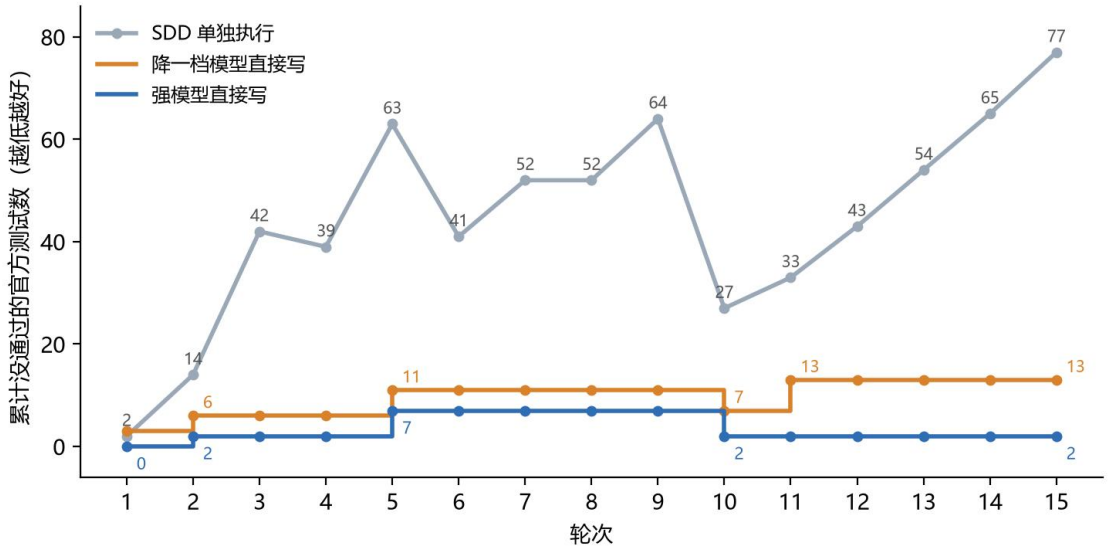


图 2 15 轮长任务中累计没通过的测试数（越低越好）

- SDD 单独执行到第 15 轮差 77 个，强模型直接写只差 2 个；逐题配对比较，从第 2 轮起强模型直接写就显著更好。所以 SDD 单独执行已不是有意义的对手，主对比应当是“同一模型，IC 对直接写”。
- 降一档模型的线是台阶形：**第 1、2、5、11 轮各新增一批错误，之后一直留着，从来没被修过**（第 10 轮的下降是因为那一轮需求改写，相关测试被删了，不是修好）。15 轮里旧功能被改坏只有 1 次。长任务的主要问题是“新错不修”，不是“旧功能退化”，**这正是回修要解决的。**
- 强模型几乎没有这个问题，管理方法没有发挥空间；降一档模型留有缺口，适合检验 IC 的价值。

观察 2：同样设置重复跑，第 1 轮的结果差别很大（模型随机波动）。

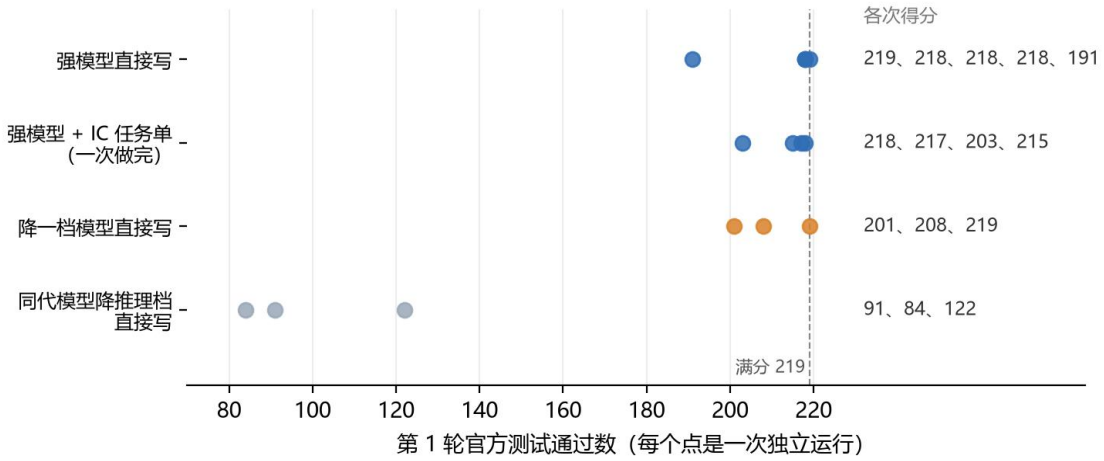


图 3 第 1 轮重复运行的官方测试通过数

- 同一个设置会偶尔整块写坏（如 191、203），单次结果不能下结论，正式对比每组都要重复跑。
- 同代模型降推理档后，每次都做到一半就停，自己承认没做完，属于“不肯做”，不适合作为对比模型。降一档模型每次都说做完了，但会漏掉原文写明的细节，属于“没做全”，正是独立检查和回修能补上的。
- 最常见的漏项是边界细节：“对数遇到 0 要返回非数值”这一条，强模型 10 次里漏了 6 次。

三、今天做出的决定

决定	依据
主对比改为：同一个降一档模型，IC 对比直接写	图 2：强模型直接写已远超 SDD 单独执行，且接近满分，比不出管理的价值；降一档模型留有缺口、更便宜
SDD 单独执行的结果冻结，不再重跑	已有完整 15 轮结果

每组重复运行，看分布而不是单次	图 3：同一设置单次得分可差 28 个测试
执行方式改为一次做完，事后检查并回修	第一、4 点的表：分步执行成本高 4.5 倍，没有收益
新增回修环节：写测试、判断、修复由不同模型分担	图 2 的“新错不修”；第一、1 和 2 点：独立检查加一次回修即可修到接近满分
预先算好期望值的检查降为辅助	只发现约一半缺陷，期望值算错还会误报

四、为什么还需要一天，明天做什么

- 开发方按今天定下的效果实现回修环节（需求已交付，正在实现）。
- 本地先用已有程序做离线验收：至少发现 13/19 处已知缺陷，满分程序上不误报，修复后不改坏任何测试。
- 正式对比：同一个降一档模型下，IC 与直接写各跑第 1 轮 3 次，再各跑完整 15 轮；只报告成功率和 token 消耗。